Análise Exploratória de Dados

Capítulo 9 — IA generativa aplicada à análise

Cayo Oliveira

Sumário

9 IA generativa aplicada à análise **3**
9.1 A resposta convincente que usou a métrica errada 3
9.2 Antes das LLMs: a AED já era parcialmente automatizada 4
9.3 Mensagens e papéis no diálogo 5
9.4 Arquitetura de software: o modelo é um componente do sistema 6
9.5 Do texto à intenção analítica 7
9.6 Métrica, dimensão e camada semântica 8
9.7 Text-to-SQL passo a passo 9
9.8 Validação independente da consulta e do resultado 11
9.9 Tokens, contexto, custo e alucinação no ponto certo 12
9.10 Copiloto, analytics conversacional e BI agentic 13
9.11 Plataformas como distribuições de responsabilidades 14
9.12 Perguntas irrespondíveis e protocolo de recusa 15
9.13 Resposta, visual e trilha de evidência 16
9.14 Roteiro de validação e fechamento 17

Capítulo 9

IA generativa aplicada à análise: da pergunta à resposta verificável

9.1 A resposta convincente que usou a métrica errada

Uma diretora comercial perguntou ao assistente do painel: “Qual região vendeu melhor no último trimestre?” Em segundos, recebeu uma resposta fluente, uma tabela e barras azuis: Nordeste, R$ 4,8 milhões, crescimento de 14%. A recomendação sugeria ampliar estoque na região. Na reunião seguinte, o financeiro mostrou outro número: R$ 3,6 milhões e queda de 2%. As duas consultas executaram sem erro. A primeira somou valor bruto, incluiu pedidos cancelados e interpretou “último trimestre” como noventa dias corridos; a segunda usou receita líquida no último trimestre civil completo.

Esse caso abre a Unidade II porque mostra a diferença entre linguagem correta e análise correta. Um modelo de linguagem consegue interpretar frases, gerar código e resumir resultados, mas não conhece automaticamente o contrato da organização. Palavras como venda, cliente, margem, ativo, melhor e recente parecem simples para pessoas do domínio, embora escondam definições, filtros, grãos e períodos. Quando a interface esconde essas escolhas, a velocidade aumenta e a contestação diminui.

O fluxo que estudaremos é: linguagem natural → intenção → métrica e dimensão → fonte e grão → SQL ou função governada → execução → validação → resposta e visual. A seta não significa que cada etapa seja gerada livremente por um LLM. Métricas podem vir de camada semântica; permissões, do sistema de identidade; consultas, de templates; cálculos críticos, de funções testadas. O modelo coordena linguagem e ferramentas, mas a confiança nasce dos contratos.

Também distinguiremos três experiências. Um copiloto auxilia uma pessoa dentro da ferramenta, por exemplo criando cálculo ou visual. Analytics conversacional recebe perguntas e devolve texto, tabela ou gráfico sobre um domínio autorizado. BI agentic planeja uma investigação com várias etapas, seleciona fontes, executa consultas, verifica hipóteses e talvez acione outro sistema. A autonomia cresce, assim como custo, latência e superfície de falha. O nome comercial não prova o nível; precisamos observar o comportamento.

Tokens, contexto, custo e alucinação aparecem somente na medida em que afetam AED. O modelo recebe texto convertido em unidades chamadas tokens dentro de uma janela de contexto. Esquema, definições, exemplos e histórico ocupam essa janela e têm custo de processamento. Contexto insuficiente favorece suposições; contexto excessivo

pode diluir informação relevante e aumentar preço e latência. Alucinação, aqui, inclui inventar coluna, função, definição ou explicação plausível que não corresponde à fonte.

Ao final do encontro, você deverá decompor uma pergunta, reconhecer ambiguidade, localizar a camada semântica, acompanhar um exemplo Text-to-SQL passo a passo, validar resultado por cálculo independente, escolher um visual coerente, distinguir assistência, conversa e agência, comparar plataformas por responsabilidades e recusar perguntas irrespondíveis. A aula dura três horas, com chamada às 20h30, e termina com um protocolo reutilizável em qualquer ferramenta.

Síntese da trilha

Regra da Unidade II. Uma resposta executável não é necessariamente correta; uma resposta correta em um recorte não é necessariamente governada; uma resposta governada ainda precisa ser comunicada com escala, fonte e limite.

9.2 Antes das LLMs: a AED já era parcialmente automatizada

Antes de existir uma conversa com modelo de linguagem, equipes já automatizavam partes da análise exploratória. Uma rotina em Python lia CSVs, verificava tipos, calculava média e quartis, contava valores ausentes, produzia histogramas e exportava um relatório. Em bancos relacionais, SQL agrupava dados por período e categoria. Em ferramentas de BI, dimensões, medidas e filtros permitiam que usuários examinassem cortes previstos por quem modelou os dados. O trabalho era automatizado, embora a pergunta e a interpretação continuassem majoritariamente humanas.

Essa automação tinha uma qualidade importante: repetibilidade. Dado o mesmo conjunto e a mesma versão de código, a função de média deveria devolver o mesmo número. Testes podiam verificar colunas, denominadores e limites. Um agendamento podia executar o pipeline todo dia e avisar se a taxa de ausência subisse. O limite estava na rigidez: uma pergunta nova podia exigir alteração de código, uma visualização nova dependia de uma pessoa que conhecesse a ferramenta, e uma métrica com definição tácita permanecia difícil de descobrir.

Auto-EDA e AutoML ampliaram a exploração automatizada. Bibliotecas podiam perfilar tabelas, sugerir gráficos, calcular associações ou comparar modelos sob um objetivo escolhido. Esses recursos economizavam operações repetitivas, mas não decidiam sozinhos qual população importava, se uma associação era causal, se a coluna significava receita bruta ou líquida, ou se o resultado era seguro para compartilhar. Automatizar a execução não é automatizar a responsabilidade analítica.

Uma LLM muda principalmente a interface de tradução e composição. A pessoa pode descrever uma intenção em linguagem natural; o sistema pode ajudar a mapear termos para métricas, gerar código, invocar ferramentas autorizadas e redigir uma história. O modelo não ganha acesso implícito aos dados, não conhece contratos ausentes e não transforma texto fluente em cálculo confiável. A arquitetura precisa encaminhar cada tarefa ao componente apropriado: recuperação para definições, SQL ou Python para quantidades e LLM para interpretação linguística limitada pelas evidências.

O quadro comparativo útil não é “velho versus novo”, mas “qual etapa é automatizada e com que garantia?”. Um script tem escopo menor e comportamento mais previsível; uma conversa aceita mais formulações, mas precisa resolver ambiguidade; um agente pode

planejar várias ferramentas, aumentando a capacidade e os caminhos de falha. Em todos os casos, a AED começa pelo problema, passa por dado e análise e termina numa decisão comunicável. O storytelling organiza a evidência; não substitui a evidência.

```
# Automação clássica: pergunta e regra foram definidas previamente.
import pandas as pd

df = pd.read_csv("atendimentos.csv")
por_canal = df.groupby("canal")["tempo_min"].agg(
    quantidade="count", mediana="median", p90=lambda s: s.quantile(.90)
)
print(por_canal.sort_values("p90", ascending=False))
```

Esse código reproduz um cálculo explícito, mas não resolve o que a gerente quer dizer por "canal pior". Pode significar maior P90, maior tempo mediano, mais reclamações ou maior diferença em relação à meta. Uma conversa pode ajudar a revelar as opções; a métrica e a comparação precisam ficar registradas antes da consulta. Na dinâmica inicial, vocês devem desenhar a cadeia e marcar o que era possível automatizar antes das LLMs.

9.3 Mensagens e papéis no diálogo

Uma chamada a um modelo de conversa não é simplesmente uma frase enviada a uma caixa de texto. O aplicativo organiza mensagens com papel, conteúdo e, em alguns protocolos, metadados ou destinatário. O papel informa como interpretar aquela parte da interação. Os nomes concretos variam entre APIs, mas a separação de origem e autoridade continua essencial para construir software previsível.

Uma instrução de sistema define objetivos e limites gerais da aplicação. Algumas interfaces expõem também um papel de desenvolvedor, destinado às regras do produto e ao contrato de saída. A mensagem do usuário contém a pergunta e, quando permitido, o contexto necessário. A mensagem do assistant é a resposta do modelo; ela também pode representar uma solicitação de ferramenta, não uma resposta final. Uma mensagem de ferramenta contém o resultado produzido pelo próprio software depois que ele valida e executa a solicitação.

Considere a pergunta "qual canal piorou?". Uma mensagem de usuário pode carregar a solicitação, mas a aplicação ainda precisa fornecer a definição de piora, o período, a tabela de evidências autorizada e o formato de saída. O modelo pode devolver uma escolha de visual ou uma narrativa estruturada. Se solicitar a função `calcular_p90(canal, periodo)`, essa solicitação não deve ser executada sem validação: a aplicação verifica se função, colunas, filtros e período pertencem à lista permitida.

O conteúdo recuperado de documentos é dado, não uma nova política de segurança. Uma página pode conter instruções maliciosas ou antigas; se o sistema a tratar como se fosse uma regra de sistema, estará confundindo evidência com autoridade. O pipeline deve manter proveniência e delimitação: instruções confiáveis vêm da aplicação, pedido vem do usuário, dado vem da fonte, e resultado de ferramenta vem de execução validada. Essa distinção reduz riscos de injeção de prompt e de instruções conflitantes.

O histórico conversacional mantém continuidade, mas não substitui estado estruturado. Depois de "agora compare com o semestre anterior", o aplicativo deve saber qual métrica, população, campus e filtro ainda valem. O estado deve poder ser exibido, revisado e

reiniciado. Se depende apenas de uma sequência extensa de mensagens, um filtro antigo pode reaparecer ou um novo turno pode herdar uma premissa que o usuário já abandonou.

Para o quadro, representem cada turno como uma sequência tipada: instruções confiáveis; solicitação; contexto recuperado com fonte; plano/saída estruturada; chamada de ferramenta validada; resultado; resposta final. Marquem com uma cor diferente o que veio do usuário, da organização, do banco e do modelo. A pergunta de checagem é simples: "Se esta parte estiver errada ou maliciosa, que componente ainda impede um número incorreto ou uma ação indevida?"

9.4 Arquitetura de software: o modelo é um componente do sistema

Uma aplicação analítica com LLM tem responsabilidades separáveis. A interface recebe a pergunta e mostra gráfico, narrativa e fontes. O serviço de aplicação organiza o fluxo e o estado. A camada de domínio contém métricas, dimensões, permissões e regras de análise. O serviço de dados consulta fontes autorizadas. Um adaptador conversa com o fornecedor do modelo. Um validador confere esquema, unidades, evidências e política antes da resposta chegar à interface.

Essa separação permite testar a AED sem chamar uma LLM. Podemos executar a função de taxa em uma amostra conhecida, conferir o denominador e verificar o gráfico. Depois testamos o adaptador com respostas simuladas e, finalmente, avaliamos o comportamento do sistema integrado. Se cálculo e narrativa estiverem dentro de um único prompt, fica difícil saber se uma falha veio dos dados, da definição, da consulta, do modelo ou da interface.

O caminho de uma solicitação é: entrada da interface → autenticação/autorização → resolução de intenção → busca de definições → plano analítico → execução por função/SQL restrito → testes e evidências → geração de narrativa → validação da resposta → exibição. Cada fronteira tem contrato. O modelo recebe apenas o necessário; cálculos críticos ficam em código determinístico; operações de escrita permanecem fora do protótipo didático ou atrás de aprovação explícita.

Um contrato de função reduz liberdade perigosa. Em vez de aceitar SQL arbitrário, a aplicação pode expor `consultar_indicador(nome_metrica, dimensao, periodo, filtros_autorizados)`. A função valida nomes cadastrados, confere acesso, cria consulta parametrizada, limita linhas e devolve resultado com metadados. O modelo pode selecionar uma combinação, mas o software continua dono da validação e da execução. O contrato também permite testar casos extremos sem depender de uma resposta linguística específica.

Segredos seguem a fronteira do servidor. A chave de API deve ser lida de variável de ambiente ou mecanismo de secrets e usada pelo backend; não deve aparecer no JavaScript do navegador, no repositório, em arquivo compartilhado ou em logs. A atividade de sala usa dados sintéticos ou públicos, sem login e sem exposição de dados pessoais. "Sem login" simplifica um protótipo didático, mas também significa que ele não está apto a proteger dados por usuário.

O custo e a confiabilidade pertencem ao desenho. Timeout, indisponibilidade, resposta malformada, excesso de chamadas e saída sem evidência são caminhos esperados, não exceções impensáveis. O aplicativo deve exibir os números calculados mesmo se o serviço linguístico falhar. Um identificador de execução permite reconstruir versão do prompt,

modelo, métrica, fonte e evidências sem registrar conteúdo sensível desnecessário.

A Figura 9.1 organiza essas responsabilidades em um fluxo para o quadro. Ela separa mensagens de usuário, instruções da aplicação, solicitações de ferramenta e resultados calculados. Observe que a execução determinística ocorre entre duas respostas do assistant; o software valida a solicitação antes de consultar dados e valida a narrativa antes de mostrá-la.

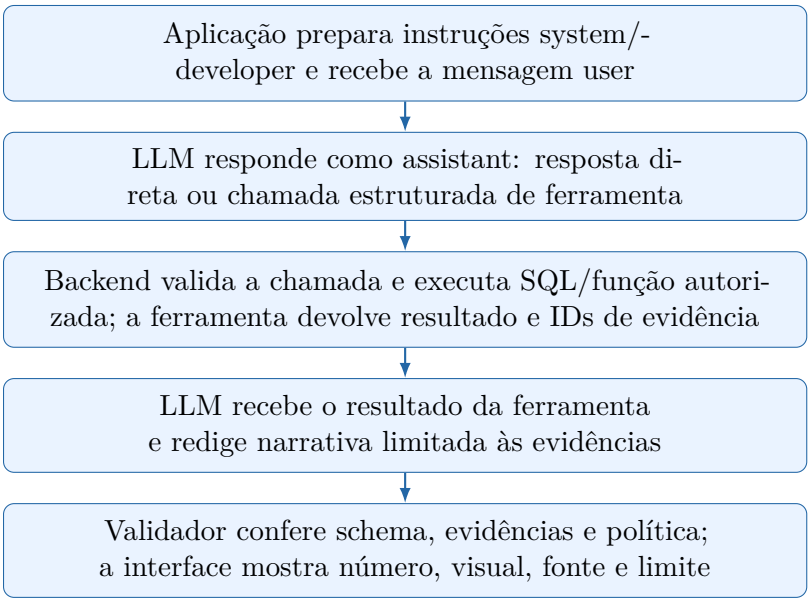


Figura 9.1: Arquitetura mínima de uma chamada a LLM com ferramenta analítica. O modelo sugere e narra; o backend autoriza e calcula; o validador controla o que chega à pessoa.

Para a primeira versão do projeto, vocês não precisam de agentes, grafo nem banco vetorial. Precisam de uma pergunta, uma pequena base compreensível, uma transformação reproduzível, um gráfico funcional, uma solicitação clara ao modelo e uma validação que impeça alegação sem suporte. O próximo capítulo detalha como transformar a pergunta em plano semântico; depois acrescentaremos recuperação de contexto quando ela realmente resolver uma lacuna.

9.5 Do texto à intenção analítica

Linguagem natural comprime decisões. “Qual região vendeu melhor?” contém ao menos quatro espaços de escolha. A intenção pode ser ranking, comparação ou diagnóstico. “Região” pode significar loja, entrega ou residência do cliente. “Vendeu” pode significar pedidos, itens, receita bruta, receita líquida ou margem. “Melhor” pode significar maior total, maior crescimento, maior margem ou maior distância da meta. O primeiro trabalho do sistema é expandir essa compressão sem inventar resposta.

Uma representação intermediária torna a intenção visível. Podemos registrar: tarefa = ranking; métrica = receita líquida; dimensão = região da loja; período = último trimestre civil completo; filtros = pedidos concluídos; granularidade de saída = uma linha por região; comparação = total e crescimento contra trimestre anterior; população = lojas autorizadas ao usuário. Essa estrutura pode ser JSON, objeto de aplicação ou formulário. O formato importa menos que a separação explícita.

Classificar intenção evita mandar toda frase direto ao gerador de SQL. Algumas perguntas pedem recuperação: “qual foi a receita?”. Outras pedem comparação, tendência, distribuição, relação, explicação de definição, criação de visual ou ação. “Por que caiu?” não é uma consulta única; requer hipóteses e várias medidas. “O que devo fazer?” exige restrições e julgamento. O roteador deve encaminhar, pedir esclarecimento ou recusar, em vez de tratar todo texto como seleção e agregação.

Ambiguidade pode ser lexical, temporal, estrutural ou decisória. “Cliente” possui sinônimos e papéis; “último mês” pode ser mês corrente incompleto ou anterior completo; “por produto” pode pedir SKU ou categoria; “melhor” não define critério. O sistema deve detectar termos com múltiplas definições cadastradas. Uma resposta que escolhe silenciosamente uma interpretação economiza uma pergunta e pode produzir uma decisão errada.

O diálogo de esclarecimento deve ser curto e orientado à consequência. Para o caso, basta perguntar: “Você quer receita líquida de pedidos concluídos por região da loja no último trimestre civil completo, comparada ao trimestre anterior?” A pessoa pode confirmar ou alterar. O sistema registra a escolha no estado da conversa e repete-a no subtítulo da resposta. Se o usuário não souber qual métrica institucional vale, a camada semântica oferece definições aprovadas em vez de pedir que improvise.

Contexto conversacional preserva referências como “e agora por canal” ou “retire Pernambuco”. Ele não substitui estado analítico explícito. Cada turno deve produzir uma intenção completa depois de resolver pronomes e elipses. Caso contrário, o histórico longo vira única fonte para reconstruir filtros. O painel precisa mostrar região, período, métrica e população atuais; a exportação precisa carregar esse estado.

Uma pergunta pode ser irrespondível por ausência de dado, definição, permissão ou desenho. “Qual campanha causou o crescimento?” não pode ser respondida por vendas agregadas sem identificação causal. “Quem comprará amanhã?” exige modelo, avaliação e finalidade. “Mostre salário individual” pode violar acesso. “Qual região é melhor?” permanece irrespondível até definir melhor. Reconhecer a condição de recusa é competência, não falha de fluência.

9.6 Métrica, dimensão e camada semântica

Métrica é uma regra de cálculo; dimensão é um eixo de agrupamento ou filtro. Uma coluna numérica não vira métrica apenas porque pode ser somada. `valor_item` pode exigir desconto, imposto, devolução, cancelamento e moeda. `regiao` pode existir em cliente, loja e endereço de entrega. A camada semântica liga termos do negócio a campos, joins, agregações, formatos e políticas, oferecendo um contrato acima do esquema físico.

No caso, definimos receita líquida como soma de valor dos itens menos desconto e devolução, apenas para pedidos concluídos, em moeda já convertida para real na camada SOT. O evento pertence à data de faturamento. A região é a região da loja responsável pela venda. O último trimestre civil completo, considerando data de referência 15/10/2026, é 01/07/2026 a 30/09/2026. Cada componente reduz liberdade do gerador.

Uma especificação compacta pode conter nome, descrição, expressão, grão, filtros obrigatórios, dimensões compatíveis, responsável e versão. Para `receita_liquida`, expressão = `valor_bruto - desconto - devolucao`; filtro = `status='CONCLUIDO'`; grão de base = item faturado; dimensões = tempo, loja, região, canal e categoria; proprietário = Controladoria; versão = 3. O LLM não precisa reconstruir essa lógica em toda pergunta.

Sinônimos ajudam a ligar linguagem ao contrato: faturamento líquido, vendas líquidas e receita após devoluções podem apontar para a mesma métrica. Mas sinônimo não deve apagar diferença real. Receita e margem não são equivalentes; pedido e item não são iguais; cliente ativo pode variar por produto. Termos conflitantes pedem esclarecimento. Um dicionário amplo sem governança apenas acelera a escolha errada.

A camada semântica também controla joins. A tabela fato possui `loja_id`; a dimensão de lojas possui uma linha por loja e região válida. Se o sistema junta uma tabela histórica com várias versões sem condição temporal, duplica vendas. O contrato define cardinalidade e chave. Em Power BI, Looker, Tableau, Quick Sight, Databricks ou código próprio, os mecanismos variam, mas a necessidade é a mesma: impedir que a frase "por região" abra um join arbitrário.

Segurança deve ser aplicada antes da resposta. Se uma gerente enxerga apenas Nordeste, o sistema não deve consultar o Brasil inteiro e depois esconder linhas no visual. A consulta precisa respeitar identidade, segurança em linha e coluna e escopo autorizado. Metadados também podem ser sensíveis; nomes de tabelas ou categorias não autorizadas não devem aparecer no prompt ou na mensagem de erro.

Camada semântica não elimina manutenção. Definições mudam, produtos surgem, fontes atrasam e metas são revistas. Cada resposta precisa guardar versão da métrica e horário. Exemplos SQL confiáveis podem envelhecer; descrições podem divergir; sinônimos podem colidir. Governança é ciclo de revisão, não documento criado uma vez. O assistente deve consultar o contrato atual e informar quando a versão não cobre a pergunta.

Elemento	Decisão no caso	Risco sem contrato
Métrica	receita líquida v3	somar valor bruto
Dimensão	região da loja	usar região do cliente
Tempo	trimestre civil completo	usar 90 dias ou mês incompleto
Status	somente concluído	incluir cancelado
Segurança	lojas autorizadas	consultar escopo indevido

9.7 Text-to-SQL passo a passo

Considere quatro tabelas. `fato_itens_venda` possui `pedido_id`, `produto_id`, `loja_id`, `data_faturamento`, `valor_bruto`, `desconto`, `devolucao` e `status`. `dim_loja` possui `loja_id`, `nome_loja` e `regiao`. `dim_produto` contém categoria e produto. `calendario` registra data, ano e trimestre. O grão da fato é um item faturado; portanto, contar linhas não conta pedidos nem clientes.

Passo 1: normalizar a pergunta. Após esclarecimento, registramos: "Rankear regiões da loja pela receita líquida de pedidos concluídos entre 01/07/2026 e 30/09/2026 e comparar com 01/04/2026 a 30/06/2026." A saída exige região, receita atual, receita anterior, diferença absoluta e variação percentual. Não pedimos causa nem previsão.

Passo 2: ligar termos ao esquema. `região da loja` aponta para `dim_loja.regiao`; `receita líquida` aponta para a expressão governada; `pedidos concluídos` para `status`; período para `data_faturamento`. A cardinalidade de `fato_itens_venda.loja_id` para `dim_loja.loja_id` é muitos-para-um. `dim_produto` não participa e deve ficar fora. Menos joins reduzem custo e risco.

Passo 3: definir limites antes de gerar. A consulta será somente leitura, executada com usuário limitado, timeout, limite de linhas e lista permitida de tabelas. Não aceitamos

INSERT, UPDATE, DELETE, funções externas ou SQL múltiplo. O sistema valida a árvore sintática e injeta o escopo de segurança. Essas regras não dependem da boa intenção do prompt.

Passo 4: gerar consulta legível. Duas CTEs calculam períodos com a mesma regra e depois se unem por região. NULLIF impede divisão por zero; COALESCE trata região presente em apenas um período. Datas usam intervalo semiaberto para evitar problemas de horário. A consulta exibe parâmetros e não inventa coluna de crescimento.

```
WITH atual AS (
  SELECT l.regiao,
         SUM(f.valor_bruto - f.desconto - f.devolucao) AS receita
  FROM fato_itens_venda f
  JOIN dim_loja l ON l.loja_id = f.loja_id
  WHERE f.status = 'CONCLUIDO'
    AND f.data_faturamento >= DATE '2026-07-01'
    AND f.data_faturamento <  DATE '2026-10-01'
  GROUP BY l.regiao
), anterior AS (
  SELECT l.regiao,
         SUM(f.valor_bruto - f.desconto - f.devolucao) AS receita
  FROM fato_itens_venda f
  JOIN dim_loja l ON l.loja_id = f.loja_id
  WHERE f.status = 'CONCLUIDO'
    AND f.data_faturamento >= DATE '2026-04-01'
    AND f.data_faturamento <  DATE '2026-07-01'
  GROUP BY l.regiao
)
SELECT COALESCE(a.regiao, p.regiao) AS regiao,
       a.receita AS receita_atual,
       p.receita AS receita_anterior,
       a.receita - p.receita AS diferenca,
       (a.receita - p.receita) / NULLIF(p.receita, 0) AS variacao
FROM atual a
FULL JOIN anterior p ON p.regiao = a.regiao
ORDER BY receita_atual DESC;
```

Passo 5: revisar antes da execução. Conferimos métrica, status, datas, join, agrupamento e ordenação. A expressão subtrai devolução já alocada no item; se devolução estiver em tabela separada, essa consulta seria errada. Conferimos tipo monetário e moeda. FULL JOIN é suportado no banco escolhido; em outro dialeto, adaptamos. SQL sintaticamente válido ainda depende do contrato físico.

Passo 6: executar e capturar evidência. Guardamos texto da pergunta, intenção estruturada, SQL, parâmetros, usuário, versão semântica, horário, duração, contagem de linhas e identificador do resultado. O modelo recebe somente o conjunto autorizado necessário para resumir. Se a consulta exceder limite ou retornar base incompleta, a resposta não segue como se fosse normal.

Passo 7: interpretar sem ultrapassar. Se Nordeste tem maior receita atual, dizemos que lidera pela definição escolhida. Se Sudeste tem maior crescimento, não o chamamos de “melhor” sem declarar critério. Uma tabela pode ordenar por receita e uma coluna

mostrar variação. O título repete período e métrica. A resposta oferece a consulta e permite contestar.

Quando a execução falha, o reparo precisa receber erro controlado e contexto mínimo. Um erro "FULL JOIN não suportado" permite reescrever com união de chaves; um erro "coluna inexistente" exige voltar ao esquema, não inventar grafia. Limitamos tentativas e comparamos a intenção estruturada após cada correção. O reparo não pode remover silenciosamente filtro, segurança ou período apenas para fazer a consulta rodar.

Quando a execução funciona, ainda pode haver reparo analítico. Um detector de anomalia pode notar que o total dobrou após o join; um teste de invariantes pode encontrar receita líquida maior que bruta. Nesse caso, o agente abre diagnóstico de grão e cardinalidade. Ele não ajusta números nem acrescenta `DISTINCT` como curativo, porque `DISTINCT` pode esconder duplicação sem restaurar a soma correta. Corrigir requer identificar a relação que produziu o fanout.

Explicação guiada

Aluno, observe a divisão de trabalho. O LLM ajuda a transformar linguagem em intenção e SQL. A camada semântica fornece significado; o banco executa; validadores impõem limites; o analista confere cálculo e visual. Nenhuma dessas responsabilidades desaparece na conversa.

9.8 Validação independente da consulta e do resultado

Validar não é perguntar ao mesmo modelo se ele está correto. Precisamos de evidência independente. O primeiro teste compara o total do SQL com uma medida certificada ou consulta de referência. Se a Controladoria publica receita trimestral de R$ 13,2 milhões e a soma regional retorna R$ 14,9 milhões, interrompemos. A diferença pode vir de status, moeda, duplicação, data ou escopo. A narrativa só começa depois da reconciliação.

Um conjunto pequeno permite cálculo manual. Suponha quatro itens do Nordeste no período: R$ 1.000 com desconto 100 e devolução 0; R$ 800 com desconto 0 e devolução 200; R$ 500 cancelado; R$ 700 com desconto 50 e devolução 0. A receita válida é $(1000 - 100) + (800 - 200) + (700 - 50) = 900 + 600 + 650 = R\$2.150$. Se a consulta retorna R$2.650, ela incluiu o cancelado.

Teste de grão detecta duplicação. Antes do join, conte itens e pedidos; depois, confira se o número de linhas cresce inesperadamente. Se `dim_loja` possui duas versões para a mesma loja e não usamos vigência, cada item pode duplicar. Compare `COUNT(*)`, `COUNT(DISTINCT pedido_id)` e soma antes/depois. Um resultado plausível não revela fanout por aparência.

Teste de período verifica fronteiras. Inclua registros em 30/06 23h59, 01/07 00h00, 30/09 23h59 e 01/10 00h00. O intervalo atual inclui julho a setembro e exclui outubro; o anterior exclui julho. Datas armazenadas com fuso exigem conversão definida. "Último trimestre" também deve permanecer igual ao refazer a resposta amanhã, por isso usamos datas materializadas na evidência.

Teste de invariantes procura propriedades. Receita por regiões deve somar o total autorizado; receita líquida não deve superar receita bruta quando devolução e desconto são não negativos; variação é nula quando a base anterior é zero; região desconhecida

deve aparecer ou ser tratada conforme contrato, não sumir por `INNER JOIN` acidental. Invariantes detectam classes de erro além de um exemplo.

Teste de segurança usa identidades diferentes. Uma gerente do Nordeste deve receber apenas lojas autorizadas; um administrador pode receber total nacional. O SQL mostrado precisa refletir ou documentar o filtro aplicado. Cache não pode entregar resultado de outro usuário. O sistema registra política e escopo sem expor detalhes indevidos.

Teste visual verifica pergunta e codificação. Para ranking, barras horizontais ordenadas ou pontos em escala comum favorecem comparação. Receita atual e anterior podem aparecer como dois pontos conectados; cor destaca somente a região escolhida. Um gráfico de setores não mostra crescimento; mapa exagera área e não é necessário se logística espacial não participa. O visual deve conservar unidade, período, base e fonte.

Por fim, teste de linguagem compara cada frase com uma célula ou cálculo. "Nordeste lidera receita líquida" aponta para maior `receita_atual`. "Cresceu 14%" aponta para fórmula e base anterior. "Deve receber estoque" não decorre automaticamente; exige margem, demanda, capacidade e prazo. O sistema pode responder ao ranking e recusar a recomendação até receber essas variáveis.

9.9 Tokens, contexto, custo e alucinação no ponto certo

Modelos de linguagem processam tokens, unidades que podem representar palavras, partes de palavras, sinais e números. Um prompt com pergunta, esquema, dicionário, exemplos e histórico consome tokens de entrada; a intenção, SQL e explicação consomem saída. O custo pode ser monetário ou computacional, conforme serviço ou modelo local. Para AED, o ponto importante é que contexto e saída não são infinitos nem gratuitos.

Enviar o esquema inteiro de um warehouse com milhares de campos é caro e confuso. Recuperação de contexto seleciona tabelas, métricas e exemplos relevantes antes do modelo. O classificador identifica domínio comercial; a busca traz `receita_liquida`, `dim_loja` e consulta certificada; campos de RH ficam fora. Esse recorte reduz exposição e melhora ligação semântica. Selecionar contexto é parte da arquitetura analítica.

Histórico conversacional também cresce. A pergunta "e por canal?" depende do turno anterior, mas mensagens antigas sobre outra análise podem contaminar o estado. Em vez de reenviar toda conversa, preservamos intenção estruturada atual, resumo auditável e referências aos artefatos. O texto humano continua disponível para interface, enquanto o executor usa um estado compacto. Isso reduz custo e ambiguidade.

Latência acumula etapas: classificar, recuperar metadados, gerar, validar, executar, talvez corrigir, recomendar visual e resumir. Um copiloto de autoria tolera interação em segundos; um agente que testa cinco hipóteses leva mais tempo. Cache ajuda em métricas estáveis, mas precisa respeitar identidade, versão e frescor. Responder rápido com dado antigo é falha, não otimização.

Alucinação em analytics pode ocorrer antes, durante ou depois do SQL. O modelo inventa coluna `receita_liquida`; escolhe `data_pedido` em vez de faturamento; cria função não suportada; interpreta zero como ausência; atribui causa ao padrão; recomenda ação sem dado. Execução captura somente algumas falhas. Uma coluna inventada falha; uma coluna existente com significado errado executa. Por isso, avaliação ponta a ponta é indispensável.

Temperatura baixa e prompt detalhado podem reduzir variação, mas não garantem

verdade. Saída estruturada limita formato, não semântica. Um JSON válido pode conter métrica errada. RAG recupera contrato, mas pode buscar versão antiga. Exemplos orientam, mas podem não cobrir o caso. Guardrails reduzem risco em camadas; nenhum substitui teste de referência e possibilidade de recusa.

Custos de consulta também importam. SQL gerado pode escanear tabela enorme, executar join cartesiano ou repetir investigação. Prévia do plano, limite de bytes, timeout, warehouse isolado, agregados e APIs governadas protegem operação. No modo agentic, um orçamento por tarefa limita número de ferramentas, consultas e tokens. O agente deve parar quando evidência suficiente ou limite é alcançado.

9.10 Copiloto, analytics conversacional e BI agentic

Copiloto é assistência situada. A pessoa permanece no fluxo de autoria ou leitura e pede ajuda para criar cálculo, gráfico, resumo ou filtro. O escopo costuma ser a fonte ou artefato aberto. Ele reduz atrito, mas não escolhe necessariamente objetivo nem conduz investigação completa. A responsabilidade de aceitar e editar fica visível. Um cálculo gerado precisa ser lido como código produzido por colega.

Analytics conversacional muda a porta de entrada. O usuário formula pergunta e recebe resultado sobre domínio preparado. O sistema resolve intenção, consulta, tabela, texto e talvez visual, mantendo contexto entre turnos. É adequado para perguntas pontuais e exploração guiada. Seu teste principal é correção ponta a ponta: a resposta corresponde à pergunta sob a semântica e permissão corretas?

BI agentic acrescenta planejamento e múltiplas ações. Diante de “por que a receita caiu?”, o agente pode decompor por região, canal, produto e tempo; escolher ferramentas; executar consultas; testar hipóteses; sintetizar evidências; citar artefatos; pedir esclarecimento e parar. Ele não deveria executar uma campanha ou alterar meta sem autorização específica. Agência analítica não implica autonomia irrestrita.

A diferença pode ser observada por controle. No copiloto, o usuário dita cada passo. Na conversa, dita a pergunta e o sistema escolhe consulta. No modo agentic, o sistema escolhe subperguntas e sequência. Quanto mais escolhas delegadas, maior a necessidade de plano visível, orçamento, logs, checkpoint, aprovação e avaliação. Um rótulo “agent” sem essas propriedades pode ser apenas chat renomeado.

Evidência produzida também muda. Um copiloto pode deixar cálculo no workbook; conversa deve mostrar fonte, filtros e SQL; investigação agentic precisa listar hipóteses, consultas, resultados rejeitados e razão do fechamento. A resposta final não pode apagar o caminho. Um relatório com citações internas permite abrir a consulta que sustenta cada afirmação.

Ferramentas não ocupam uma única caixa. Uma plataforma pode oferecer autoria visual, copiloto, chat e agentes diferentes. Versões e licenças variam. Portanto, compare a tarefa concreta: quem define métrica, quem seleciona fonte, onde a segurança atua, como consulta é mostrada, quem escolhe visual, como corrigir e quanto custa. A marca é contexto de implementação, não categoria pedagógica.

O protocolo de recusa cresce com autonomia. Copiloto pode dizer que não cria o cálculo sem definição. Conversa pode pedir período. Agente pode interromper investigação quando fontes discordam, orçamento acaba, permissão falta ou causalidade não pode ser estabelecida. Um sistema que sempre responde parece útil até a primeira decisão cara.

Experiência	Escolha delegada	Evidência mínima
Copiloto	execução de passo solicitado	cálculo ou visual editável
Conversacional	intenção e consulta pontual	métrica, filtros, consulta e resultado
BI agentic	plano, subperguntas e ferramentas	plano, traces, consultas, testes e limites

9.11 Plataformas como distribuições de responsabilidades

Tableau Agent ajuda a explorar a fonte conectada, criar visualizações e cálculos, filtrar e ordenar. A documentação atual ressalta escopo e limites: ele não é chatbot aberto, trabalha na fonte selecionada e depende de campos, descrições e dados preparados. O ponto para AED é verificável: quando a ferramenta cria um visual, examine agregação, período, campo e escala. Tableau Next amplia a conversa para semântica e experiências agentic, mas continua dependente de contratos e permissões.

No Power BI, Copilot atende consumidores e autores sobre relatórios e modelos semânticos; também pode gerar DAX. Fabric data agents e experiências agentic distribuem outras responsabilidades. Um DAX compilável não garante definição correta. O modelo semântico, medidas, relacionamentos e segurança formam a base. A pergunta de avaliação é qual artefato foi usado e se estava preparado para IA.

Amazon Q em Amazon Quick Sight oferece Generative BI para perguntas, autoria, resumos e histórias. O domínio pode ser organizado por Topics, instruções e datasets. O nome histórico QuickSight Q ainda aparece em conversas, mas o material usa a nomenclatura atual. A lição não é decorar menus: linguagem natural funciona melhor quando o domínio foi curado e a identidade restringe o que pode ser consultado.

Looker Studio e Looker não são equivalentes. Looker Studio favorece relatórios conectados mais acessíveis; Looker oferece LookML, Explores, governança e APIs. Gemini em Looker sustenta analytics conversacional e agentes de dados sobre essa semântica. O contrato em LookML pode impedir que "receita" vire soma improvisada, desde que definições e joins estejam corretos e mantidos.

Databricks AI/BI Genie Agent combina Unity Catalog, SQL warehouse, instruções, exemplos e ativos confiáveis. No modo de investigação, pode planejar e executar várias consultas. Isso aproxima BI agentic, mas amplia a necessidade de conferir hipótese, fonte, custo e citações. Catálogo controla acesso e descoberta; não escolhe sozinho o denominador certo.

Python e Streamlit deixam a arquitetura exposta. Podemos construir roteador, recuperação semântica, banco somente leitura, validador SQL, tabela, gráfico e trilha de evidência. Essa transparência é valiosa para aprendizagem e protótipo. Em contrapartida, catálogo, RLS, auditoria, escala e administração não surgem automaticamente. Liberdade significa implementar e testar cada responsabilidade.

Uma comparação justa não pergunta qual plataforma "tem mais IA". Pergunta onde residem métrica, dimensão, permissão, execução, visual, log, custo e contestação. Uma organização pode combinar dbt ou warehouse para métricas, Power BI para distribuição e Streamlit para caso especializado. Contratos entre componentes importam mais que a promessa de uma interface única.

Para comparar implementações, use o mesmo conjunto de perguntas e resultados de referência. Inclua uma pergunta direta, uma ambígua, uma com sinônimo, uma que cruza

períodos, uma sem permissão, uma causal e uma de base zero. Registre se o sistema esclarece, qual consulta produz, se respeita o escopo, quanto demora e se cita evidência. Uma demonstração bem-sucedida escolhida pelo fornecedor mede conveniência, não robustez.

Separe ainda qualidade da resposta e qualidade da experiência. Duas ferramentas podem produzir o mesmo número, mas uma mostra SQL e filtros enquanto outra oferece somente texto; uma pode permitir correção local e outra exigir manutenção do modelo semântico; uma pode ter menor latência e maior custo operacional. A escolha depende de governança, equipe e consequência. O capítulo ensina a formular essa avaliação, não a declarar um vencedor universal.

9.12 Perguntas irrespondíveis e protocolo de recusa

Há perguntas irrespondíveis por dados. “Qual cliente está insatisfeito?” não pode ser deduzida apenas de atraso sem pesquisa ou proxy validada. “Por que a margem caiu?” exige decomposição e talvez informação de custos ausente. O sistema identifica campos necessários e diz o que falta. Inventar explicação para preencher silêncio é alucinação aplicada.

Há perguntas irrespondíveis por ambiguidade. “Melhor vendedor” pode usar receita, margem, meta, retenção ou qualidade. Se as definições levam a rankings diferentes e o usuário não escolhe, o sistema apresenta opções e pede esclarecimento. Não deve selecionar a que produz história mais interessante. A recusa pode ser temporária e informativa.

Há perguntas proibidas por acesso ou finalidade. Um usuário pode pedir dados individuais, atributo sensível ou comparação fora de seu território. O sistema não revela existência de colunas protegidas nem tenta contornar. Explica que não possui autorização e oferece agregado permitido quando apropriado. A política atua na ferramenta e na consulta, não apenas na frase final.

Há perguntas que exigem causalidade. “A campanha aumentou vendas?” não se resolve por coincidência temporal. Se existe experimento ou estratégia de identificação, o agente pode chamar análise apropriada. Sem isso, responde que observa associação e propõe quais dados ou desenho seriam necessários. Transformar “depois” em “por causa” é uma das falhas mais perigosas da narrativa automática.

Há perguntas cujo custo excede benefício. Uma investigação pode pedir centenas de consultas sobre dados enormes. O agente estima plano e orçamento, usa agregados, pede recorte ou interrompe. Tokens, tempo e computação fazem parte da decisão. Limite de custo não deve virar resultado incompleto apresentado como completo.

O protocolo de recusa contém motivo, evidência disponível, lacuna e próximo passo. “Não posso afirmar qual região foi mais lucrativa porque a fonte autorizada contém receita, mas não custo alocado. Posso comparar receita líquida ou solicitar a visão certificada de margem.” Essa resposta preserva utilidade sem fingir. Também registra a tentativa para melhorar catálogo ou documentação.

Recusar não é sempre terminar. O sistema pode decompor: responder parte segura, marcar parte incerta e pedir confirmação. Porém, precisa separar claramente o que foi calculado do que foi sugerido. Uma interface que mistura ambos em um parágrafo fluente impede contestação. Tabela, fonte, aviso e pergunta de esclarecimento podem ocupar blocos distintos.

Explicação guiada

Condição de recusa. Pare quando faltar definição, dado, permissão, validade causal, orçamento ou mecanismo de validação. Diga o motivo e qual informação permitiria continuar.

9.13 Resposta, visual e trilha de evidência

Depois da validação, a resposta deve começar pela interpretação escolhida: “Considerando receita líquida v3, pedidos concluídos, região da loja e trimestre civil completo de julho a setembro de 2026...”. Essa frase parece longa, mas evita que o número circule sem contrato. Em seguida vem o resultado, a comparação e o limite. Definições detalhadas podem ficar expansíveis.

A tabela apresenta valores exatos; o visual favorece padrão. Barras horizontais ordenam receita atual. Um ponto menor ou coluna adjacente mostra trimestre anterior. Se o foco é crescimento, um gráfico de inclinação pode ser melhor. Não use mapa apenas porque há região; a pergunta é ranking comercial, não localização. O sistema de NL2Vis precisa justificar a codificação.

O título deve refletir o que a consulta provou. “Nordeste lidera receita líquida no 3ž trimestre; Sudeste tem maior crescimento” diferencia critérios. “Nordeste é a melhor região” reintroduz ambiguidade. Cor destaca uma descoberta; escala inicia em zero para barras; moeda e período aparecem; regiões com base anterior zero recebem indicação, não percentual infinito.

A trilha de evidência acompanha resposta. Ela inclui pergunta original, esclarecimento, intenção estruturada, métrica e versão, dimensões, filtros, SQL ou API, horário, usuário, resultado, testes executados e visual. Nem todo usuário verá tudo aberto, mas deve conseguir acessar o nível necessário. Auditores e desenvolvedores recebem logs mais profundos, respeitando segurança.

Feedback deve ser específico. “Errado” pode significar definição, dado, SQL, visual ou linguagem. A interface oferece categorias e comentário. Correção de um usuário não altera métrica institucional automaticamente; gera proposta revisada pelo responsável. Caso contrário, uma conversa local pode reprogramar o significado para todos.

Respostas precisam expirar. Se a fonte atualiza diariamente, uma captura de ontem deve mostrar horário. Se a métrica muda de versão, resultados anteriores permanecem identificados. Cache possui chave com usuário, pergunta normalizada, versão semântica, período e fonte. Reprodutibilidade inclui tempo.

Dataviz continua relevante porque externaliza comparação. A frase do modelo é serial: chega em uma ordem escolhida por ele. Um gráfico permite ver simultaneamente todas as regiões, distâncias e exceções, contestar o destaque e procurar padrão não mencionado. Conversa orienta; visual inspeciona; tabela confirma; trilha reproduz. As quatro formas se complementam.

Uma resposta compartilhável deve preservar esse conjunto. Copiar apenas o parágrafo para uma mensagem remove filtro, visual e versão; capturar somente o gráfico remove consulta e esclarecimento. A exportação profissional reúne título, tabela ou visual, fonte, atualização, definição e identificador da trilha. Assim, outra pessoa consegue abrir o artefato original e verificar se o contexto continua válido.

9.14 Roteiro de validação e fechamento

O primeiro movimento é reescrever a pergunta como contrato. Identifique intenção, métrica, dimensão, período, filtros, população, comparação e ação. Marque termos ambíguos. Consulte a camada semântica e peça esclarecimento quando interpretações mudarem a decisão. Se não houver definição aprovada, não deixe o LLM criar uma silenciosamente.

O segundo movimento é restringir execução. Selecione apenas esquema relevante, aplique identidade, use conexão somente leitura, valide SQL, limite custo e registre parâmetros. Verifique grão, joins, status e datas antes de executar. Não envie dados sensíveis ao modelo quando basta enviar agregados.

O terceiro movimento é validar de forma independente. Compare total certificado, calcule amostra manual, teste fronteiras, invariantes, duplicação e segurança. Confira resultado em tabela. Se uma diferença não puder ser explicada, pare. Tentar um novo prompt até aparecer número esperado é viés de confirmação automatizado.

O quarto movimento é comunicar. Escolha título calibrado, tabela, visual e nota. Relacione cada frase a evidência. Separe resultado, interpretação e recomendação. Mostre filtros, período, fonte, atualização e versão. Ofereça SQL ou artefato equivalente. Verifique acessibilidade e escala como na Unidade I.

O quinto movimento é decidir entre responder, esclarecer ou recusar. Responda quando contrato, acesso, dado e validação são suficientes. Esclareça quando opções legítimas existem. Recuse quando falta base, permissão, causalidade ou orçamento. Em todos os casos, explique próximo passo possível.

Durante as três horas, o primeiro bloco percorre intenção, semântica e ambiguidade; o segundo acompanha Text-to-SQL e validação manual; após a chamada, o terceiro compara níveis de assistência, plataformas, custos e recusas; o fechamento monta resposta e trilha. O mesmo caso impede que o encontro vire catálogo de recursos.

A abertura da Unidade II muda a pergunta profissional. Antes, perguntávamos qual gráfico construir. Agora perguntamos quais decisões foram delegadas, quais contratos restringem o modelo e que evidência permite contestar. O LLM reduz esforço de tradução e código; não elimina definição, estatística, arquitetura nem percepção. Ele aumenta o valor de quem consegue verificar a cadeia.

No próximo encontro, aprofundaremos a ligação de esquema, NL2Vis e consultas governadas. O fluxo de hoje já fornece a espinha dorsal: linguagem natural, intenção, semântica, execução, validação e visual. A profundidade seguinte estará nos casos difíceis, não na repetição da promessa de conversar com dados.

Síntese da trilha

Síntese final. Pergunta → intenção → contrato semântico → consulta restrita → execução → validação independente → resposta, visual e trilha. Fluência acelera; evidência autoriza.

Referências essenciais

PHEBE. *A Multi-Agent Natural Language Interface for Interactive Data Visualization.* ACM IAIT, 2026. DOI: https://doi.org/10.1145/3816713.3819784.

SINGH et al. *Beyond Text-to-SQL: An Agentic LLM System for Governed Enterprise Analytics APIs.* arXiv, 2026. Disponível em: https://arxiv.org/abs/2605.21027.

EDGE et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization.* arXiv:2404.16130, 2024.

OPENAI. *Responses API overview, function calling e tools*. Documentação oficial consultada em 20 out. 2026: https://developers.openai.com/api/docs/guides/tools.

Documentação oficial consultada em 20 out. 2026: Microsoft Fabric data agents e modelos semânticos; Tableau Next/Agentforce Tableau; Amazon Quick Generative BI; Gemini Conversational Analytics em Looker; Databricks AI/BI Genie. Recursos, nomes, regiões, licenças e disponibilidade mudam; conferir a documentação antes de uma demonstração.